外设驱动、摄像头配置与固化

嵌入式对接教程（二）

2026 FPGA 创新设计赛道（安路）赛题二

嵌入式方向 • 04_embedded

读者：嵌入式方向队员（零基础，建议先读第一册）
目标：把状态送出板子（UART）、把摄像头配起来（I2C）、把 bit 烧进去（QSPI）
前置要求：会 Verilog、跑过 ModelSim；了解 TD 软件界面
预计学习时间：8～12 天（每天 2～3 小时）

这一册解决什么

” 从 bit 到能演示” 的最后一段路：三个外设通道（UART 遥测、按键）、一个必需的外设配置（摄像头寄存器表）、一条可复现的固化流程（TD 工程 + QSPI）。

目录

第一部分　从零学习教程 **3**

1　你在造什么 **3**
1.1　一句话 . . . 3
1.2　三个外设通道 . . . 3
1.3　做完是什么效果 . . . 3

2　阶段 0：UART 串口原理（1～2 天） **3**
2.1　一根线的协议 . . . 3
2.2　波特率与分频系数 . . . 4
2.3　练习 . . . 4

3　阶段 1：遥测协议设计（1～2 天） **4**
3.1　9 字节一包 . . . 4
3.2　”快照 + 发送中跳过”策略 . . . 4
3.3　练习 . . . 5

4　阶段 2：I2C 与摄像头配置（2～3 天） **5**
4.1　两条线的协议 . . . 5
4.2　摄像头寄存器表 . . . 5
4.3　练习 . . . 5

5　阶段 3：按键消抖（1 天） **5**
5.1　机械抖动的物理现实 . . . 6
5.2　练习 . . . 6

6　阶段 4：TD 工具链与固化（2～3 天） **6**
6.1　工程组织与文件清单纪律 . . . 6
6.2　TCL 全流程骨架 . . . 6
6.3　固化 QSPI：让它”上电即运行” . . . 7
6.4　练习 . . . 7

7　阶段 5：跑通外设测试（1 天） **7**
7.1　两个测试 . . . 7

第二部分　工程实现详解 **8**

8　工程总览 **8**
8.1　外设层文件清单 . . . 8
8.2　外设层的验证策略 . . . 8

9 模块详解：uart_telemetry **8**
9.1 bit_idx 状态机（v2，重写版） 9
9.2 组帧快照与跳过 9
9.3 波特率分频与误差预算 9

10 模块详解：uart_rx_mon（TB 侧监视器） **10**
10.1 两个关键设计 10
10.2 v1 的失败：内联监视器漏沿 10

11 模块详解：i2c_bfm_sc500 **10**
11.1 开漏总线的建模 10
11.2 位相处理与记录 10

12 配置链测试与遗留（诚实的记录） **11**
12.1 tb_i2c_config 的” 对录” 方法 11
12.2 遗留：ACK 位相未收敛 11

13 固化 runbook 详解 **11**
13.1 filelist.f 的三段式 11
13.2 引脚约束增量 12
13.3 固化五步与冷启动验收 12

14 风险清单与交接 **13**

15 两册总览 **13**

结语 **13**

总结与下一步 **14**
本册学到了什么 14
下一步做什么 14

从零学习教程

1 你在造什么

1.1 一句话

给这块板子接上”嘴”（UART 遥测）、”耳朵”（按键）、”眼睛的说明书”（摄像头配置），并让它一上电自己跑起来（固化）。

1.2 三个外设通道

```
UART 遥测:  FPGA --(115200 串口)--> 电脑/上位机 (每帧一包目标状态)
按键:       KEY0 -> 切阈值组    KEY1 -> 跟踪使能开关 (经官方消抖)
I2C:        FPGA --(SCL/SDA, 0x36)--> SC500CS 摄像头 (116 条寄存器)
固化:       TD 综合 -> bit -> BitWizard 写 QSPI Flash -> 上电即运行
```

1.3 做完是什么效果

插上 USB 串口，串口助手每 16.7ms 收到一包 9 字节数据（`A5 01 1C 02 53 01 00 F1 5A` 这样的东西），解析出”已锁定、坐标 (540, 339)”；按 KEY0 红绿蓝阈值组轮流切换；拔掉电脑，板子独立上电，15 秒内出图并进入跟踪。

学完本册你能做到

- 手算 UART 波特率分频，说出 8N1 帧的每一位
- 设计带定界与校验和的遥测协议，并解释”发送中跳过”策略
- 讲清 I2C 的 START/ACK/STOP 与寄存器写事务格式
- 读懂 SC500CS 720p60 寄存器表与分辨率变更的连锁清单
- 按 runbook 走完 TD 工程、引脚、固化、五步联调检查点

2 阶段 0：UART 串口原理（1～2 天）

2.1 一根线的协议

UART 没有时钟线，靠**双方约定的波特率**同步。一帧 8N1：

```
空闲 = 高电平
起始位：1 拍低          <- 接收方看到下降沿，知道" 要来了"
数据位：8 位，LSB 先发
停止位：1 拍高
(无校验位：N = None)
```

接收方在起始沿之后，每隔**一个位时间**采一次样；采在位的正中间最稳（抗抖动）。

2.2 波特率与分频系数

FPGA 没有”波特率发生器”，只有系统时钟，用分频凑：

```
位时间 = CLK_DIV / f_clk
CLK_DIV = 646, f_clk = 74.25MHz
实际波特率 = 74.25e6 / 646 = 114,938   (目标 115200, 误差 -0.23%)
```

误差多大算安全

UART 容忍总误差 ±2～3%（收发各占一半）。0.23% 有十倍余量，随便换时钟都能接受。校验方法：**实际值 / 目标值 - 1**，绝对值小于 2% 就上。

2.3 练习

1. 手算：50MHz 时钟下，115200 波特率的分频系数是多少？误差多少？
2. 画出一帧字节 `0xA5` 在线上（txd）的电平序列（含起止位）
3. 思考：如果时钟有 5% 误差、连发 9 个字节，为什么后几个字节更容易错？

3 阶段 1：遥测协议设计（1～2 天）

3.1 9 字节一包

偏移	内容	说明
0	A5	包头（定界）
1	valid	目标有效位
2	cx 低 8 位	
3	cx 高 3 位	(11bit 坐标拆成两个字节)
4	cy 低 8 位	
5	cy 高 3 位	
6	preset	当前阈值组号
7	checksum	前 7 字节低 8 位累加和
8	5A	包尾（定界）

三个设计点：**双字节定界**（A5 开头 5A 结尾，上位机扫描同步头）；**累加和校验**（一行代码，挡住所有单字节错误）；**坐标拆高低字节**（11bit 装不进 1 字节，低 8 + 高 3 是标准做法）。

3.2 ”快照 + 发送中跳过”策略

每帧（16.7ms）触发一次组帧，但一包要发 780us——

```
帧拍到达 && 不在发送中 -> 快照当前坐标，开始发
帧拍到达 && 正在发送   -> 跳过本次（下一帧再发）
发送期间一切不变 -> 绝不打断当前包
```

为什么不做发送 FIFO

”跳过”策略保证任意时刻板子只发**完整包**，上位机不需要处理残包。代价是发送忙时丢帧——但 780us 远小于 16.7ms 帧周期（占用率约 5%），实际几乎不丢。**能用策略解决的，不要用缓冲解决**——少一个 FIFO 少一份面积和验证。

3.3 练习

1. 手算：9 字节 x 10 位 x 8.68us/位约等于多少？一帧发送占用率是多少？
2. 写一个 Python 上位机解析伪代码（找 A5、收 9 字节、验证 5A 与校验和）
3. 思考：如果想加一个” 帧号” 字段，应该插在哪两位之间？为什么？

4 阶段 2：I2C 与摄像头配置（2～3 天）

4.1 两条线的协议

I2C 只有 SCL（时钟）和 SDA（数据），且都是**开漏**：谁都不会主动拉高，只能拉低或放手；”1” 由**上拉电阻**提供。

```
START:  SCL 高电平期间，SDA 由高变低
STOP:   SCL 高电平期间，SDA 由低变高
数据:   SCL 低电平期间变化，高电平期间稳定（接收方在上升沿采）
ACK:    第 9 个时钟，从机把 SDA 拉低表示" 收到"
```

整条 I2C 最容易错的地方：位相

”SCL 高时 SDA 下降” 既是 START 的定义，也是**数据位变化**的违规情形。从机状态机必须把” 数据位” 和” 起始/停止” 分清，否则会出现” 地址识别正确、ACK 却总被判成 START” 的诡异现象（本项目真实遇到过，见 Part II 遗留一节）。

4.2 摄像头寄存器表

SC500CS 的配置就是一张**寄存器写入清单**：720p60 配置共 **116 条**，写事务格式：

```
{0x6C, reg_hi, reg_lo, data}      <- 4 字节一事务
0x6C = 7bit 地址 0x36 左移一位 + 写方向位 0
(地址 0x36 是 SC500 系列出厂默认)
```

表存在 ROM 里（`uics500reg_720p60.v`，官方提供），配置引擎（`uicfgcs500_720p` + `uii2c`）逐条读出并发送。

换分辨率不是” 换一张表” 那么简单

官方 ROM 文件头写着一份连锁清单：MIPI 接收参数、图像宽高、ISP、DDR 帧缓存、HDMI 时序、HDMI TX 参数、时钟与时序约束——**全部要用同一组分辨率帧率**，否则黑屏/花屏/错位。本项目锁定 720p60 不动，改分辨率要走接口变更流程。

4.3 练习

1. 手算：7bit 地址 0x36 左移一位，加写位（0）后是多少？
2. 画出写事务 `6C 03 5A 12` 在 SDA/SCL 上的完整波形（含两次 ACK）
3. 思考：为什么 I2C 用开漏而不是推挽？（提示：多设备并联）

5 阶段 3：按键消抖（1 天）

5.1 机械抖动的物理现实

按键内部是金属弹片，按下/松开瞬间会**弹跳几毫秒**——电平在 0/1 之间抖十几下。直接计数会一次按下多切好几档。

```
官方消抖：key_remove_shakes
    I_key_in （低有效取反后进） -> 内部连续采样，稳定若干 ms 才认
    O_key_trig_out: 一次按键 = 一个脉冲 （不是电平!）
用法：if (key0_trig) preset_idx <= preset_idx + 1;   // 按一下加一
```

触发脉冲 vs 电平的分工

消抖模块输出的是**单拍脉冲**（” 按下” 这个事件），不是” 按住” 电平。所以顶层用 if 直接触发动作，不需要自己再写边沿检测——**把边沿检测封在消抖模块里**，顶层逻辑只剩一行。

5.2 练习

1. 估算：74.25MHz 下消抖 10ms 需要多少拍？
2. 思考：如果按键接的是” 按住就连续加” 的旋钮场景，脉冲输出还适用吗？

6 阶段 4：TD 工具链与固化（2～3 天）

6.1 工程组织与文件清单纪律

```
td_prj/ 工程主文件 (prj.al)
src/    全部 RTL (按 scripts/filelist.f 加入)
filelist.f 三段式:
  (1) 视觉方向新增 (13 个 rtl 文件)
  (2) 嵌入式新增 (system_top / uart_telemetry)
  (3) 仿真专用 -- 禁止加入 TD 工程 (替身/TB/sim_video_src 等)
```

filelist 的” 第三段” 是保命的

`divider_gate_sim.v`、`ph1p_logic_eram_stub.v` 这些替身文件如果被顺手加进 TD 工程，会和真网表/真原语**抢同一个模块名**——综合结果随机、上板行为不可预期。把” 禁止加入” 写进清单文件本身，让它在最显眼的地方。

6.2 TCL 全流程骨架

```
prj_create -name vision_ptz -device PH1P35MDG324 -force
prj_add_files -file_list scripts/filelist.f
prj_set_constraint -sdc ... -adc ...
compile -impl syn_1     ;# 综合
map     -impl syn_1     ;# 映射
pnr     -impl syn_1     ;# 布局布线
bitgen -impl syn_1      ;# 出 .bit
rep_timing -impl syn_1 ;# 看 WNS >= 0
rep_resource -impl syn_1
```

TCL 命令名要在 TD 里逐条核对

上面的骨架按 TD 6.2.1 的习惯编写，各版本命令名/参数有差异——**首跑时打开 TD 的 Tcl 控制台，用实际补全逐个核对**。骨架的价值是流程顺序（综合-映射-布线-bit 的依赖关系），不是命令拼写。

6.3 固化 QSPI：让它”上电即运行”

```
1. TD 工具 -> BitWizard -> 目标器件选板载 QSPI Flash
2. 加载 syn_1/vision_ptz.bit -> Boot Image Tool 转换/烧写
3. 烧写完成 -> 断电 -> 拔 USB -> 重新上电
4. 15 秒内应出图并进入跟踪（参数全部固化在 bit 里）
5. 验收" 拔 PC 运行": 显示器独立供电，无 PC 参与
```

为什么必须固化（赛事硬约束）

赛道要求”冷启动直接跟踪”：评委上电就是演示状态，不允许插电脑加载。这也反过来说明为什么视觉参数（阈值/增益/限幅）全部做成 RTL parameter——**参数进 bit，上电即生效**。

6.4 练习

1. 打开 TD，用 Tcl 控制台验证骨架里的 `compile` 命令实际签名
2. 列出要把 O_pwm_x/y 和按键加到 pin 约束时需要的三样信息（脚位、电平标准、原理图页码）
3. 思考：Boot Image Tool 转换前后文件大小变化的原因

7 阶段 5：跑通外设测试（1 天）

7.1 两个测试

```
--- tb_uart_telemetry ...
    注入 3 帧 (valid=1/0/1, 坐标各异) -> 解码串行字节流
    # FRAME OK valid=1 cx=540 cy=339 sum OK
    ==> PASS        <- 全部帧正确且含 >=2 帧 LOCK

--- tb_i2c_config ...
    uicfgcs500_720p + uii2c + ROM 对着 I2C BFM 跑
    # CFG cmp 116/116 OK
    ==> 部分通过 (ACK 位相遗留, 见 Part II)
```

本册学习清单（全部打勾就过关）

- 能手算 UART 分频与误差，画出 8N1 波形
- 能解释遥测协议的定界/校验/拆字节三要素
- 能画出 I2C 写事务波形并说明 ACK 位相陷阱
- 能说清 filelist 第三段的作用与分辨率变更的连锁清单
- 按 runbook 完成一次固化与五步联调检查点
- 跑通 tb_uart_telemetry，说清 tb_i2c_config 遗留的根因

工程实现详解

这一部分写什么

外设驱动与配置链逐模块讲透：uart_telemetry 的 bit_idx 状态机与 v1 重写、uart_rx_mon 的”自由运转”监视器、i2c_bfm_sc500 的开漏建模与 ACK 位相遗留、配置链 TB 的对录方法，以及固化 runbook 逐节的执行细节。本册对应迭代记录的第 4、5、6 轮。

8 工程总览

8.1 外设层文件清单

文件	类型	行数	职责
`rtl/uart_telemetry.v`	RTL	94	目标状态遥测（115200-8N1）
`tb/uart_rx_mon.v`	仿真	64	UART 接收监视器（两处复用）
`tb/tb_uart_telemetry.v`	测试	88	3 帧注入逐字节验证
`rtl/i2c_bfm_sc500.v`	仿真	88	SC500CS I2C 从机 BFM
`tb/tb_i2c_config.v`	测试	93	配置链对录（116 条）
`sim/uics500reg_720p60.v`	官方	—	116 条寄存器 ROM
`sim/uii2c.v / uicfgcs500_720p.v`	官方	—	I2C 控制器与配置引擎
`scripts/filelist.f`	脚本	—	TD 源文件清单（三段式）
`scripts/td_build_and_flash.md`	文档	—	固化 runbook

8.2 外设层的验证策略

通道	验证方法	状态
UART 遥测	自收自解（写-> 线上-> 监视器解码）逐字节比对	PASS
摄像头配置	官方配置链 vs BFM 记录，逐事务对录 ROM	116/116 一致，位相遗留
按键	顶层 glue 一行逻辑，随整链仿真覆盖	已覆盖

为什么 UART 可以”自收自解”而 I2C 不行

UART 是**点对点**：自己发的自己收，闭环即可证明字节正确——监视器是”独立的第二双眼睛”（独立实现，不共享发送端代码路径）。I2C 是**总线**：主控行为（位相/仲裁/时钟拉伸）需要有从机应答才能走完流程，所以必须写一个 BFM 当对手方——而 BFM 自身的正确性又需要真机波形佐证（这正是遗留项的由来）。

9 模块详解：uart_telemetry

9.1 bit_idx 状态机（v2，重写版）

发送状态机把”位时间”当节拍：

```
if (div_cnt == CLK_DIV - 1) begin          // 一个位时间到
    div_cnt <= 0;
    case (bit_idx)
        4'd0: begin  O_txd <= 1'b0;                  // 起始位
                     shifter <= payload[byte_idx]; end // 装载
        4'd9: begin  O_txd <= 1'b1;                  // 停止位
                     if (byte_idx == NBYTES-1) sending <= 1'b0;
                     else byte_idx <= byte_idx + 1; end
        default: begin O_txd <= shifter[0];          // 数据位 LSB 先行
                       shifter <= {1'b0, shifter[7:1]}; end
    endcase
    bit_idx <= (bit_idx == 4'd9) ? 4'd0 : bit_idx + 4'd1;
end else div_cnt <= div_cnt + 10'd1;
```

v1 的失败：装载与移位双写

第一版把”装载 payload”和”移位输出”写在同一拍的两个分支里，装载时移位的状态互相覆盖，波形全乱。重写原则：**一个节拍只干一件事**——bit_idx=0 装载（不发数据），1..8 只移位，9 只出停止位。状态机的每个状态职责单一，就没有”两件事打架”的空间。

9.2 组帧快照与跳过

```
tick_d <= I_frame_tick;                        // 边沿检测
if (I_frame_tick && !tick_d && !sending) begin
    payload[0] <= 8'hA5;
    payload[1] <= {7'd0, I_target_valid};
    payload[2] <= I_cx[7:0];   payload[3] <= {5'd0, I_cx[10:8]};
    payload[4] <= I_cy[7:0];   payload[5] <= {5'd0, I_cy[10:8]};
    payload[6] <= {5'd0, I_preset};
    payload[7] <= 8'hA5 + ...（前 6 字节累加和）;
    payload[8] <= 8'h5A;
    sending <= 1'b1;
end
```

`tick_d` 边沿检测保证”每帧只触发一次”（即使 frame_tick 因跨域被拉宽）；`!sending` 门控实现跳过策略。校验和在同一拍用组合加法算出来——这拍只有 7 个 8bit 加法，摆在帧边界上，时序零压力。

9.3 波特率分频与误差预算

```
CLK_DIV = 646                      // 参数, 74.25e6 / 115200 = 644.5
实际 = 74125000..? 按 74.25e6 / 646 = 114,938 bps
误差 = 114938 / 115200 - 1 = -0.23%   (容差 ±2~3%)
```

注意这里选的是” 整数分频取最接近值”，不追求精确整除——**波特率不需要精确，只需要在容差内**。若换 50MHz 时钟，重算 CLK_DIV 即可（参数化设计，一处改全局生效）。

10 模块详解：uart_rx_mon（TB 侧监视器）

10.1 两个关键设计

```
// 1) 两级同步器：串行输入跨进本域
sync <= {sync[0], I_rxd};   wire rxd = sync[1];

// 2) 半位时间后采样，对齐位中心
if (!rxd && !busy) begin busy <= 1; div <= CLK_DIV/2; ... end
// 3) 假起始重同步：起始位中心仍为高 -> 判定假起始，立刻退出
```

” 半位对齐” 是 UART 接收的标准姿势：检测到起始**下降沿**后等半个位时间再开始按整位采样，采样点就落在每个位的**中心**，左右各有半位余量——最抗时钟误差和线上毛刺。

10.2 v1 的失败：内联监视器漏沿

第一版在 TB 里内联写监视代码：`@(negedge txd)` 等待起始沿、再阻塞式延迟采数据位。结果字节值逐位错乱——

事件同步 + 阻塞延迟的经典坑

`@(negedge)` 只对” 当前时刻之后” 的边沿敏感；而” 包内” 的下一位起始沿可能正好发生在阻塞延迟的**窗口内部**，被错过之后状态机再也对不齐。修复：重写为**自由运转状态机**（每拍看电平、按计数器推进），没有任何 @ 事件同步——**时钟节拍是唯一的同步源**。这个教训和 Step3 的 tb_pwm 重写（任务式同步 -> 连续监视器）是同一课。

一个监视器，两处复用

`uart_rx_mon` 既用在 tb_uart_telemetry（验证遥测模块本身），也用在 tb_system_integration（验证整链里的遥测通道）。独立实现 + 两处复用 = 第三次交叉验证（发送端 FSM、接收监视器、整链调用者三种视角）。

11 模块详解：i2c_bfm_sc500

11.1 开漏总线的建模

```
assign I_sda = pull ? 1'bz : 1'b0;   // 只能拉低或放手
// TB 侧: pullup(cam_sda);           // "1" 由上拉提供
```

BFM 不主动驱动高电平——” 放手”（z）后由 TB 的上拉（1kΩ 行为模型）拉高。这是**忠实建模开漏**的唯一正确方式；若用 `1'b1` 直接驱动，主控和从机同拍驱动相反值时会得到 X，仿真掩盖真实故障。

11.2 位相处理与记录

```
wire scl_rise = I_scl && !scl_d;      // 上升沿: 采样/ACK
wire scl_fall = !I_scl && scl_d;      // 下降沿: 数据变化/计数
```

```
wire start_c  = I_scl &&  sda_d && !I_sda;   // SCL 高, SDA 下降
wire stop_c   = I_scl && !sda_d &&  I_sda;   // SCL 高, SDA 上升
// 第 9 个上升沿: 地址匹配 -> 拉低 ACK;  第 9 个下降沿: 释放
// 地址匹配: 移位寄存器 == 8'h6C (7bit 0x36 左移 + 写位)
```

记录数组放在模块内部（`reg [7:0] O_bytes [0:MAX_BYTES-1]`），TB 用层次化引用读取——**Verilog-2001 的端口列表不允许数组**，这是绕开限制的标准做法（和 system_top 的遥测层次化引用同源）。

12 配置链测试与遗留（诚实的记录）

12.1 tb_i2c_config 的”对录”方法

```
官方配置引擎 uicfgcs500_720p + uii2c  --驱动--> SCL/SDA
                                                 |
                                    BFM 记录全部写事务字节
                                                 v
期望 ROM uics500reg_720p60 --------------------> 逐条比对
判据: cfg_done 拉高 且 前 116 条写事务 (addr,reg_hi,reg_lo,data)
      与 ROM 完全一致 (AE 追加写只计数不校验)
```

这个方法的价值：即使 ACK 位相有争议，**主控侧发出的寄存器序列本身**已经被独立证明正确（从 ROM 侧逐条确认）——把”哪一部分可信、哪一部分待定”切得干干净净。

12.2 遗留：ACK 位相未收敛

现象与根因

464 次地址识别**全部正确**（BFM 侧地址匹配逻辑工作正常），但每字节之后状态被**自身的 START 检测误判清零**——BFM 在第 9 个 SCL 上升沿才拉低 SDA 做 ACK，而”SCL 高时 SDA 下降”正是 START 的定义，于是 ACK 被自己的 START 逻辑吃掉。两轮位相修复未收敛（改早了波形对不上，改晚了被判 START）。

处置：明确列为 TD 阶段项——上板用逻辑分析仪抓真机 SCL/SDA 波形对照，半天可收敛；寄存器序列本身已由 ROM 侧确认，**外设层的”配置正确性”不依赖这个未收敛项**。

无硬件条件下的正确姿势

一个 BFM 位相问题卡住两轮后，最优解不是继续盲试，而是：**记录根因假设 + 明确所需证据（真机波形）+ 判定对交付的影响（不影响）**。工程交付允许带”已知且已界定”的遗留项，不允许带”说不清”的问题。

13 固化 runbook 详解

13.1 filelist.f 的三段式

```
# (1) 视觉方向新增: 13 个 rtl (ser96to24 ... vision_top, font_rom)
# (2) 嵌入式新增:  uart_telemetry.v, system_top.v
```

```
# (3) 仿真专用 -- 禁止加入 TD 工程:
#     *_stub.v / *_sim.v (替身), sim_video_src.v, frame_dump24.v, tb/*
```

第（1）段还定义了”哪些视觉文件进综合”：三个生成文件（`hsv_inv_lut.v`、`font_rom_8x16.v`、`marquee_rom_64.v`）在内，仿真专用模块不在内。

13.2 引脚约束增量

```
set_pin_assign {O_pwm_x} -loc {XX} -io_type LVCMOS33
set_pin_assign {O_pwm_y} -loc {XX} -io_type LVCMOS33
set_pin_assign {I_btn_n[0]} -loc {XX} -io_type LVCMOS33
set_pin_assign {I_btn_n[1]} -loc {XX} -io_type LVCMOS33
# 约束文件新增线网: O_pwm_x / O_pwm_y / I_btn_n[1:0] / O_uart_txd
# clock 约束照例程 SDC 不动
```

脚位从 40Pin DC3 的空闲脚里挑（对照原理图），电平统一 3.3V。**舵机信号线接 PWM 脚，舵机电源必须独立 5V 供电并共地**。

13.3 固化五步与冷启动验收

步骤	要点
1. 打开 BitWizard	目标器件选**板载 QSPI Flash**（不是 FPGA SRAM）
2. 加载 bit 并转换	Boot Image Tool，生成 Flash 镜像
3. 烧写	用下载器写 Flash
4. 断电重上电	拔 USB，验证”无 PC 参与”
5. 15 秒内出图	出图 + 进入跟踪（参数已在 bit 里）

五步联调节奏（每步过了才下一步）

（1）例程原样跑通出图 ->（2）挂 detect_top：画面无变化 + ChipWatcher 见 t_valid ->（3）挂 OSD：出现 TRK0 文字与框 ->（4）接舵机独立电源：duty 变化时画面稳定 ->（5）固化 + 冷启动验收。

这个顺序的设计逻辑是**从”官方已知好”逐级加入变量**，每级只引入一个新东西——出错时责任面永远是一步之宽。

14 风险清单与交接

风险	对策
舵机供电	独立 5V 共地；画面随舵机动作抖动 = 供电不合格（仿真测不出）
AE 拉动颜色	演示前锁曝光，或用 8 组阈值现场切换（视觉已备）
时序未收敛	例程已知现象；挂视觉后重跑时序报告（视觉 <3% LUT，预期无碍）
tuser 链	上板”有数据无帧同步”先查 demosaic tuser（第一册界碑）
ACK 位相	逻辑分析仪抓真机波形，半天收敛（本册遗留项）

本册交付清单

- `uart_telemetry.v`：遥测模块（PASS）+ 复用监视器
- `i2c_bfm_sc500.v` + 对录式配置链 TB（116/116 序列确认）
- `filelist.f` 三段式清单与 `td_build_and_flash.md` runbook
- 五步联调检查点与四条风险对策（交接给上板阶段）
- 一个边界清晰的遗留项（ACK 位相）+ 收敛路径

15 两册总览

交付层	主张	证据
契约层	视觉兼容 96bit 契约	视觉回归 14 项
挂载层	全链可共存（elaboration 零错）	system_top
行为层	数据流贯通（受 demosaic 限制）	tb_system_integration
外设层	遥测逐字节正确	tb_uart_telemetry
配置层	寄存器序列 116 条一致	tb_i2c_config（位相待收敛）
固化层	上电即运行	runbook + 五步检查点（上板执行）

嵌入式对接的两册到这里

第一册管”装进去、跑起来”，第二册管”送出去、配起来、烧进去”。两册的任务边界就是上板阶段的工作清单——带着这两册和 `README_ 分工与移交.md` 的勾选表上板，每一步都有据可查。

结语

总结与下一步

本册学到了什么

- UART 8N1 原理、波特率分频与误差预算
- 遥测协议三要素（双字节定界/累加和/坐标拆字节）与跳过策略
- I2C 开漏总线、START/ACK/STOP 与寄存器写事务
- SC500CS 720p60 寄存器表（116 条）与分辨率连锁清单
- 按键消抖的触发脉冲用法
- TD 工程组织、filelist 纪律、TCL 流程与 QSPI 固化 runbook
- 五步联调检查点与四条风险提示

下一步做什么

- **上板首战**：按五步检查点逐级点亮（例程 -> 检测 -> OSD -> 舵机 -> 固化）
- **收敛遗留项**：用逻辑分析仪对照 tb_i2c_config 的 ACK 位相，半天可收敛；恢复 demosaic tuser 链的 TD 侧验证
- **联合验收**：按 README_ 分工与移交.md 的勾选表，每过一项打一个勾，作为过程证据存档